Data Structures and Algorithms - Final Assignment

Question 1: Graph Theory - Dijkstra's Algorithm (25 Points)

a) Explain the theoretical foundation of Dijkstra's algorithm for finding the shortest path in a weighted graph. Discuss why it fails when negative weight edges are present.

b) Trace the execution of Dijkstra's algorithm on a graph with vertices {A, B, C, D, E} and edges (A,B,4), (A,C,2), (B,C,5), (B,D,10), (C,E,3), (E,D,4), (D,A,7). Assume 'A' is the source node. Show the state of the distance array and the priority queue at each step.

c) Provide a Python implementation of Dijkstra's algorithm using a priority queue (heapq). Analyze both its time and space complexity.

Question 2: Dynamic Programming - The Knapsack Problem (25 Points)

a) Define the 0/1 Knapsack problem. Formulate this problem mathematically, clearly defining the objective function and the constraints.

b) Write the Bellman equation (recurrence relation) that solves the 0/1 Knapsack problem. Explain the base cases and the recursive step in detail.

c) Given a knapsack with a maximum weight capacity W = 50, and three items: Item 1 (Weight: 10, Value: 60), Item 2 (Weight: 20, Value: 100), Item 3 (Weight: 30, Value: 120). Use dynamic programming to determine the maximum value that can be placed in the knapsack. Show the DP table.

Question 3: Advanced Tree Structures - AVL Trees (25 Points)

a) What is the defining property of an AVL tree? How does this property guarantee O(log n) search, insertion, and deletion times?

b) Describe the four types of rotations used to maintain balance in an AVL tree: Left-Left (LL), Right-Right (RR), Left-Right (LR), and Right-Left (RL). Provide a small example for each where the rotation becomes necessary.

c) Insert the following sequence of numbers into an initially empty AVL tree: 10, 20, 30, 40, 50, 25. Show the tree after each insertion and indicate any rotations performed.

Question 4: System Design and Concurrency (25 Points)

a) Explain the concept of a Deadlock in operating systems. What are the four Coffman conditions that must hold simultaneously for a deadlock to occur?

b) Describe the Bakery Algorithm for mutual exclusion. How does it ensure fairness (no starvation) among competing threads?

c) Design a thread-safe Singleton class in Java or Python. Explain how you ensure that only one instance is created even in a highly concurrent environment. Discuss the 'Double-Checked Locking' idiom.